

The logo for Oracle Academy. The word "ORACLE" is in a bold, orange, sans-serif font. Below it, the word "Academy" is in a smaller, dark gray, sans-serif font. The entire logo is centered on a light gray background, which is framed by dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Programming with PL/SQL

7-4

Recognizing the Scope of Exceptions

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

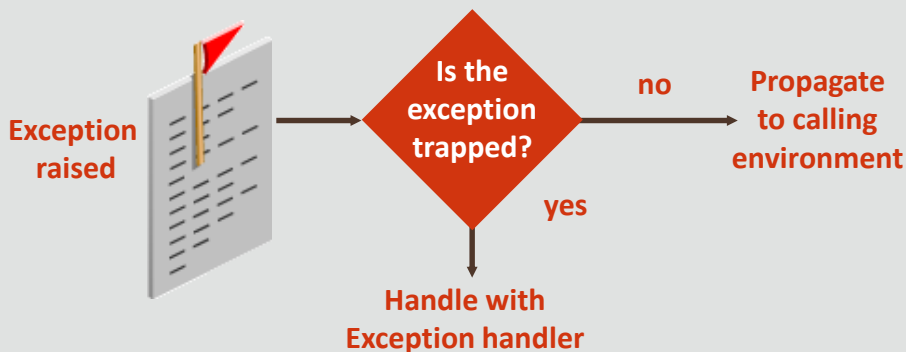
- This lesson covers the following objectives:
 - Describe the scope of an exception
 - Recognize an exception-scope issue when an exception is within nested blocks
 - Describe the effect of exception propagation in nested blocks

Purpose

- You learned about nested blocks and scope of variables in an earlier lesson
- An exception is a PL/SQL variable; therefore, it follows the same scoping and visibility rules as any other kind of variable
- To handle exceptions correctly, you must understand the scope and visibility of exception variables
- This is particularly important when using nested blocks

Exception Handling in Nested Blocks

- You can deal with an exception by:
 - Handling it (“trapping it”) in the block in which it occurs, or
 - Propagating it to the calling environment (which can be a higher-level block)



ORACLE
Academy

PLSQL 7-4
Recognizing the Scope of Exceptions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

5

A PL/SQL block nested within another PL/SQL block may be called a nested block, an enclosed block, a child block, or a sub-block.

A PL/SQL block that calls another PL/SQL block, anonymous or named, may be referred to as either the enclosing block or the parent block.

Handling Exceptions in an Inner Block

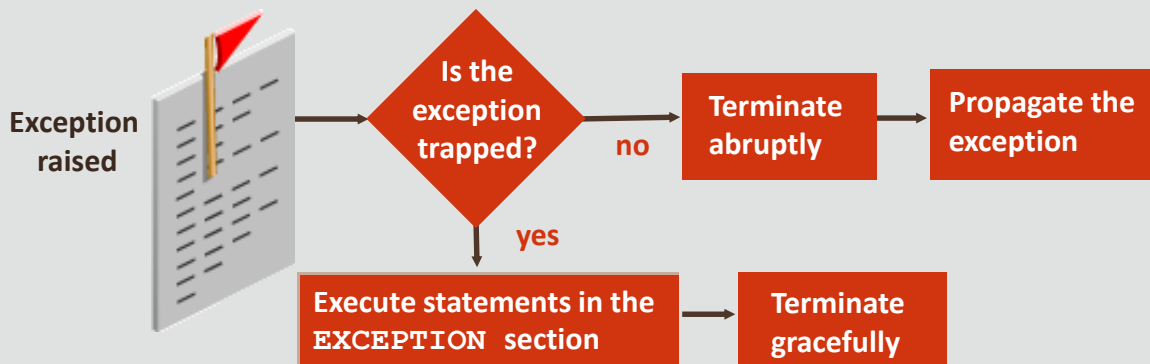
- In this example, an error occurs during the execution of the inner block
- The inner block's EXCEPTION section deals with the exception successfully, and PL/SQL considers that this exception is now finished
- The outer block resumes execution as normal

Handling Exceptions in an Inner Block

```
BEGIN -- outer block
...
BEGIN -- inner block
... -- exception_name occurs here
...
EXCEPTION
  WHEN exception_name THEN -- handled here
  ...
END; -- inner block terminates successfully
... -- outer block continues execution
END;
```

Propagating Exceptions to an Outer Block

- If the exception is raised in the executable section of the inner block and no corresponding exception handler exists, the PL/SQL block terminates with failure and the exception is propagated to an enclosing block



The general advantage of a nested block is that you create a scope for all the declared objects and executable statements in that block. You can use this scope to improve your control over activity in your program. This also means that if an error occurs in a block, then it is just in this block execution is halted, once the error has been handled locally, program control continues outside that block.

Propagating Exceptions to an Outer Block

- In this example, an exception occurs during the execution of the inner block
- The inner block's EXCEPTION section does not deal with the exception

Propagating Exceptions to an Outer Block

```
DECLARE      -- outer block
  e_no_rows  EXCEPTION;
BEGIN
  BEGIN      -- inner block
    IF ... THEN RAISE e_no_rows; -- exception occurs here
    ...
  END;       -- Inner block terminates unsuccessfully
  ...       -- Remaining code in outer block's executable
  ...       -- section is skipped
EXCEPTION
  WHEN e_no_rows THEN - outer block handles the exception
  ...
END;
```

Propagating Exceptions to an Outer Block

- The inner block terminates unsuccessfully and PL/SQL passes (propagates) the exception to the outer block
- The outer block's EXCEPTION section successfully handles the exception

Propagating Exceptions to an Outer Block

```
DECLARE      -- outer block
  e_no_rows  EXCEPTION;
BEGIN
  BEGIN      -- inner block
    IF ... THEN RAISE e_no_rows; -- exception occurs here
    ...
  END;       -- Inner block terminates unsuccessfully
  ...       -- Remaining code in outer block's executable
  ...       -- section is skipped
EXCEPTION
  WHEN e_no_rows THEN - outer block handles the exception
  ...
END;
```

Propagating Exceptions from a Sub-Block

- If a PL/SQL raises an exception and the current block does not have a handler for that exception, the exception propagates to successive enclosing blocks until it finds a handler
- When the exception propagates to an enclosing block, the remaining executable actions in that block are bypassed
- One advantage of this behavior is that you can enclose statements that require their own exclusive error handling in their own block, while leaving more general exception handling (for example WHEN OTHERS) to the enclosing block
- The next slide shows an example of this

Propagating Predefined Oracle Server Exceptions from a Sub-Block

- Employee_id 999 does not exist
- What is displayed when this code is executed?

```
DECLARE
    v_last_name      employees.last_name%TYPE;
BEGIN
    BEGIN
        SELECT last_name INTO v_last_name
        FROM employees WHERE employee_id = 999;
        DBMS_OUTPUT.PUT_LINE('Message 1');
    EXCEPTION
        WHEN TOO_MANY_ROWS THEN
            DBMS_OUTPUT.PUT_LINE('Message 2');
    END;
    DBMS_OUTPUT.PUT_LINE('Message 3');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Message 4');
END;
```

ORACLE
Academy

PLSQL 7-4
Recognizing the Scope of Exceptions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

14

Answer : Since employee_id 999 does not exist, the SELECT statement within the inner block results in an error and reverts to the local exception handling section. Because the error is not handled there, the error propagates to the calling environment. In this case, the calling environment is the outer block. The WHEN OTHERS exception handler gets executed and the output is "Message 4."

Propagating User-named Exceptions: Ex. 1

- What happens when this code is executed?

```
BEGIN
  DECLARE
    e_myexcep    EXCEPTION;
  BEGIN
    RAISE e_myexcep;
    DBMS_OUTPUT.PUT_LINE('Message 1');
  EXCEPTION
    WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Message 2');
  END;
  DBMS_OUTPUT.PUT_LINE('Message 3');
EXCEPTION
  WHEN e_myexcep THEN
    DBMS_OUTPUT.PUT_LINE('Message 4');
END;
```

ORACLE
Academy

PLSQL 7-4
Recognizing the Scope of Exceptions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

15

Answer: An error, E_MYEXCEP is not declared in the outer block.

Scope of Exception Names

- Predefined Oracle server exceptions, such as `NO_DATA_FOUND`, `TOO_MANY_ROWS`, and `OTHERS` are not declared by the programmer
- They can be raised in any block and handled in any block
- User-named exceptions (non-predefined Oracle server exceptions and user-defined exceptions) are declared by the programmer as variables of type `EXCEPTION`
- They follow the same scoping rules as other variables



Scope of Exception Names

- Therefore, a user-named exception declared within an inner block cannot be referenced in the exception section of an outer block
- To avoid this, always declare user-named exceptions in the outermost block



Propagating User-named Exceptions: Ex. 2

- Now what happens when this code is executed?

```
DECLARE
    e_myexcep    EXCEPTION;
BEGIN
    BEGIN
        RAISE e_myexcep;
        DBMS_OUTPUT.PUT_LINE('Message 1');
    EXCEPTION
        WHEN TOO_MANY_ROWS THEN
            DBMS_OUTPUT.PUT_LINE('Message 2');
    END;
    DBMS_OUTPUT.PUT_LINE('Message 3');
EXCEPTION
    WHEN e_myexcep THEN
        DBMS_OUTPUT.PUT_LINE('Message 4');
END;
```

ORACLE
Academy

PLSQL 7-4
Recognizing the Scope of Exceptions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

18

Answer: "Message 4" is displayed.

Propagating User-named Exceptions: Ex. 3

- What happens when this code is executed?

```
DECLARE
  e_myexcep      EXCEPTION;
BEGIN
  BEGIN
    RAISE e_myexcep;
    DBMS_OUTPUT.PUT_LINE('Message 1');
  EXCEPTION
    WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Message 2');
  END;
  DBMS_OUTPUT.PUT_LINE('Message 3');
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Message 4');
END;
```

ORACLE
Academy

PLSQL 7-4
Recognizing the Scope of Exceptions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

19

Answer: since the "User-Defined Exception" is not handled by the code, it is returned to the calling environment (in our case APEX).

Propagating Unhandled Exceptions to the Calling Environment

- If a raised exception is not handled in a block, the block is exited with the exception still raised
- If there is an enclosing block for the current block, the exception is passed on to that block
- The enclosing block then becomes the current block
- If a handler for the raised exception is not found, the process repeats
- If no handler is found in any block, the calling environment, for example Application Express, must then try to handle the exception



Propagating Unhandled Exceptions to the Calling Environment

- Because Application Express is Oracle software and therefore understands PL/SQL exceptions, Application Express will display an error message
- But other applications cannot always do this, and may fail with unexpected errors
- To avoid this, always handle exceptions within PL/SQL
- One way to guarantee this is to always include a WHEN OTHERS handler in the outermost block



Terminology

- Key terms used in this lesson included:
 - Exception scope
 - Exception visibility
 - Propagation of exceptions

- Exception Scope – The portion of a program in which the exception is declared and is accessible.
- Exception Visibility – The portion of the program where the exception can be accessed without using a qualifier.
- Propagation of exceptions – The inner block terminates unsuccessfully, and PL/SQL passes the exception to the outer block.

Summary

- In this lesson, you should have learned how to:
 - Describe the scope of an exception
 - Recognize an exception-scope issue when an exception is within nested blocks
 - Describe the effect of exception propagation in nested blocks

The Oracle Academy logo is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy